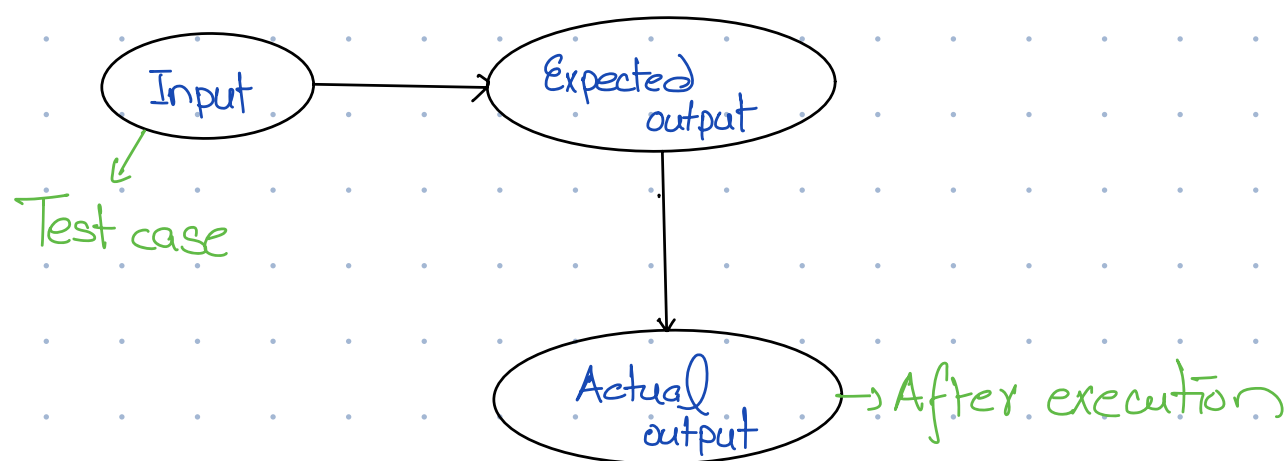


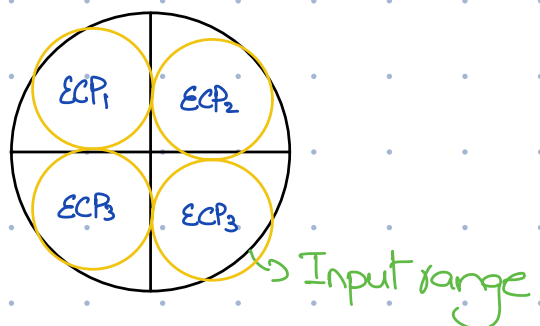
Black Box Testing



There are two techniques to perform black box testing:

1. Equivalence class participation (ECP)

We can't test all possible inputs. So, we divide them into groups (classes) & choose one test case from each group.



Example,

Let's say we have an input field whose valid range is 18-30.

ECP₁: 18-30

ECP₂: <18

ECP₃: >30

We create a table as:

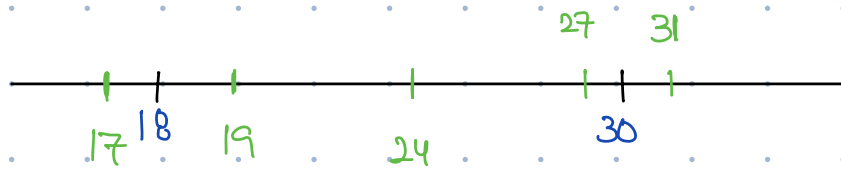
Testcase ID	Description	Input	EC	Expected Output
TC_01	Check standard valid value	20	1	Accept
TC_02	Check below range	15	2	Reject
TC_03	Check above range	35	3	Reject

2. Boundary Value Analysis

Check values near the boundary of valid input.

Example,

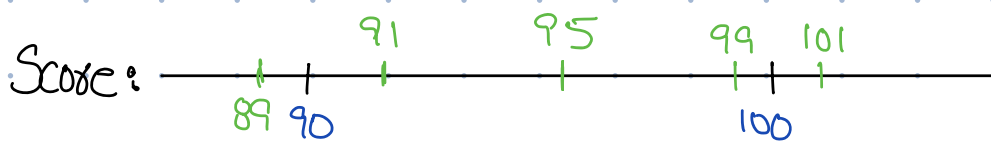
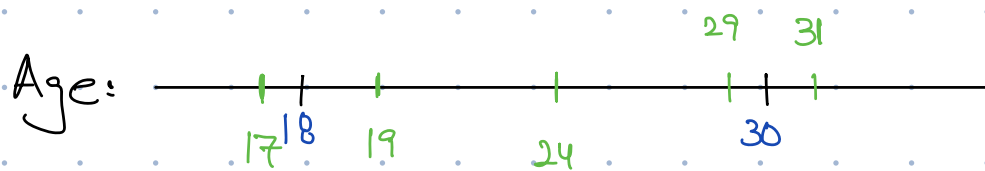
- Assume same example as above:



We choose our testcases.

- For testing two input fields we ^{to it Average} fix some & vary others in combinations.

Lets say we have two input fields as:



(varied)
Age

(fixed)
Score

17

95

19

95

24

95

27

95

30

95

Now, (fixed)
Age

(varied)
Score

24

89

24

91

24

95

24

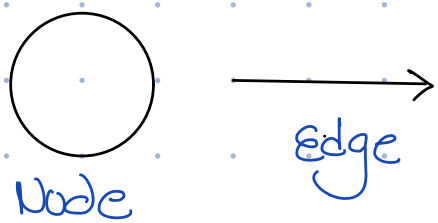
99

In case we have 3 input fields, we fix two, vary one & then fix one, vary two.

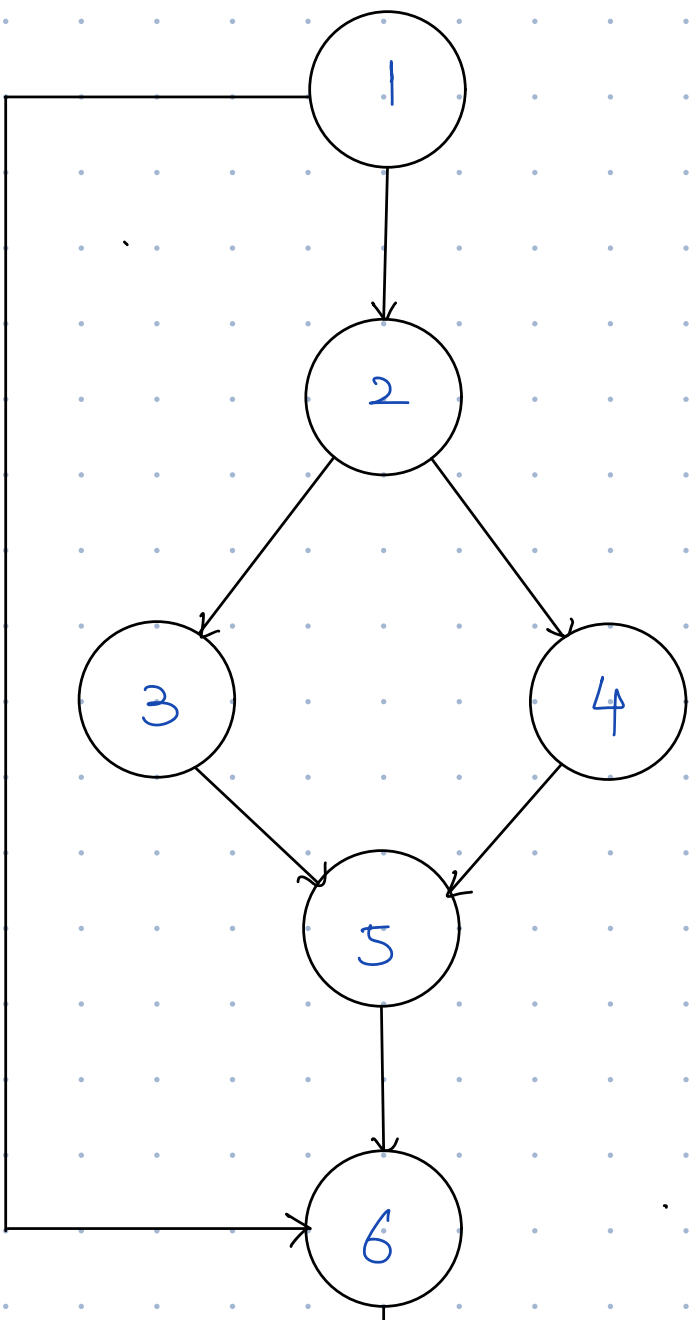
White Box Tes

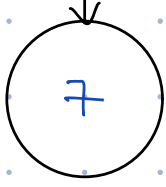
```
1 if A=10 then
2   if B>C
3     A=C
4   else A=B
5   end if
6 end if
7 print ABC
```

We express this code as Control Flow Graph (CFG)



For above code, we have:





Cyclomatic complexity tells complexity wdc & paths. In above code, we have following paths:

$P_1: 1, 2, 3, 5, 6, 7$

$P_2: 1, 2, 4, 5, 6, 7$

$P_3: 1, 6, 7$

Cyclomatic Complexity = Edges - Nodes + 2

OR

No. of closed regions + 1

For each path, we create a test case that causes that path to execute.

We compute below metrics:

1. Statement Coverage

The Goal: Ensure every single line of code (statement) is executed at least once.

Metric: $\frac{\text{Number of executed statements}}{\text{Total number of statements}} \times 100\%$

2. Branch Coverage (Decision Coverage)

The Goal: Ensure every decision point (like `if`, `while`, `case`) is evaluated to both **True** and **False** at least once.

$\frac{\text{Number of executed branches}}{\text{Total number of branches}} \times 100\%$

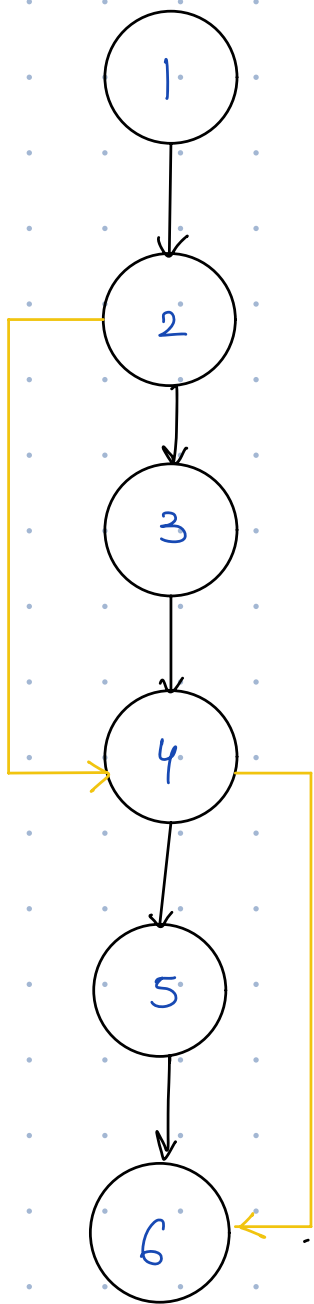
3. Path Coverage

The Goal: Ensure every possible **independent route** through the program (from Start to End) is executed.

$\frac{\text{Number of executed paths}}{\text{Total number of paths}} \times 100\%$

```
1. INPUT X, Y
2. IF X > 0:
3.     PRINT "X is Positive"
4. IF Y > 0:
5.     PRINT "Y is Positive"
6. return
```

CFC:



Possible Paths:

1, 2, 3, 4, 5, 6

1, 2, 4, 5, 6

1, 2, 4, 6

1, 2, 3, 4, 6

CC: 3

Test case 1:

$$X=1, Y=1$$

Executed path: 1, 2, 3, 4, 5, 6

Executed statements: 6

Evaluated branches: 2

$$\text{Path coverage} = \frac{1}{4} \times 100 \\ = 25\%$$

$$\text{Statement coverage} = \frac{6}{6} \times 100 \\ = 100\%$$

$$\text{Branch coverage} = \frac{2}{4} \times 100 \\ = 50\%$$

We can stop now. As, we have hit 100%
Statement & branch coverage

Test case 3:

$$X=1, Y=-1$$

Executed path: 1, 2, 3, 4, 6

Executed statements: 5

Evaluated branches: 2

$$\text{Path coverage} = \frac{3}{4} \times 100 \\ = 75\%$$

$$\text{Statement coverage} = \frac{6}{6} \times 100 \\ = 100\%$$

$$\text{Branch coverage} = \frac{4}{4} \times 100 \\ = 100\%$$

Test case 2:

$$X=-1, Y=-1$$

Executed path: 1, 2, 4, 6

Executed statements: 4

(Not added because only unique are counted)

Evaluated branches: 2

$$\text{Path coverage} = \frac{2}{4} \times 100 \\ = 50\%$$

$$\text{Statement coverage} = \frac{6}{6} \times 100 \\ = 100\%$$

$$\text{Branch coverage} = \frac{4}{4} \times 100 \\ = 100\%$$

Test case 4:

$$X=-1, Y=1$$

Executed path: 1, 2, 4, 5, 6

Executed statements: 5

Evaluated branches: 2

$$\text{Path coverage} = \frac{4}{4} \times 100 \\ = 100\%$$

$$\text{Statement coverage} = \frac{6}{6} \times 100 \\ = 100\%$$

$$\text{Branch coverage} = \frac{4}{4} \times 100 \\ = 100\%$$

